Polimorfismo

Ing. Andrés Eduardo Paternina Ariza Esp.
Universidad del Magdalena
apaternina@unimagdalena.edu.co

¿Dónde estábamos?

En las sesiones anteriores construimos:

- **Definición de Clase y Objeto**
- **Abstracción**
- **Encapsulamiento**
- **Herencia**
- **Relaciones y colaboración entre objetos (Composición)**

Los cuatro pilares de la POO

Abstracción

Quedarse con lo esencial del dominio y dejar fuera el resto.

Encapsulamiento

Proteger los datos del objeto y permitir su acceso solo a través de sus métodos.

Herencia

Crear clases nuevas a partir de una existente y reutilizar sus atributos y métodos.

Polimorfismo

¿Qué es polimorfismo?

Un **mismo método** se comporta **de forma distinta** según el objeto que lo ejecuta.

Sobrescritura (override)

Cada clase hija redefine el método heredado a su manera.

Misma llamada, distinta respuesta

Se sabe qué retorna el método, pero no cómo cada objeto lo resuelve por dentro.

Enunciado

El hotel **Costa Dorada** necesita desarrollar un sistema para administrar la información de sus trabajadores. Actualmente cuenta con tres tipos de empleados: **recepcionistas, personal de limpieza y gerentes.**

De cada trabajador se necesita almacenar su **número de identificación, nombre completo, salario básico y fecha de contratación.** Sin embargo, cada tipo de trabajador tiene características y responsabilidades particulares.

Recepcionista

- Tiene asignado un idioma principal de atención.
- Atiende huésped

Personal de limpieza

- Tiene asignada una zona del hotel.
- Limpia Habitación

Gerente

- Tiene asignado un departamento.
- Autorizar Gasto

Todos los trabajadores deben poder **consultar su información básica incluyendo sus atributos únicos** y conocer su **antigüedad en años** dentro del hotel.

Primera parte: el problema

Recepcionista	PersonalLimpieza	Gerente
Consultar información()	Consultar información()	Consultar información()
Calcular antigüedad()	Calcular antigüedad()	Calcular antigüedad()
Atender huésped()	Limpiar habitación()	Autorizar gasto()

Primera parte: el problema

Recepcionista	PersonalLimpieza	Gerente
Consultar información()	**Consultar información()**	**Consultar información()**
Calcular antigüedad()	**Calcular antigüedad()**	**Calcular antigüedad()**
Atender huésped()	Limpiar habitación()	Autorizar gasto()

Clase padre: Empleado

```
public class Empleado
{
    // Atributo: _identificacion
    // Propiedad: Identificacion
    public Empleado(string identificacion)
    {
        Identificacion = identificacion;
    }
    public virtual void ConsultarInformacion()
    {
        Console.WriteLine($"Identificación: {Identificacion}");
    }
    public void CalcularAntiguedad() { }
}
```

Clase hija: Recepcionista

```
public class Recepcionista : Empleado
{
    // Atributo: _idioma
    // Propiedad: Idioma

    public Recepcionista(string identificacion, string idioma)
        : base(identificacion)
    {
        Idioma = idioma;
    }

    public override void ConsultarInformacion()
    {
        Console.WriteLine($"Identificación: {Identificacion}");
        Console.WriteLine($"Soy recepcionista y hablo {Idioma}");
    }
    public void AtenderHuesped() { }
}
```

Clase hija: PersonalLimpieza

```
public class PersonalLimpieza : Empleado
{
    // Atributo: _zona
    // Propiedad: Zona

    public PersonalLimpieza(string identificacion, string zona)
        : base(identificacion)
    {
        Zona = zona;
    }

    public override void ConsultarInformacion()
    {
        Console.WriteLine($"Identificación: {Identificacion}");
        Console.WriteLine($"Soy personal de limpieza de la {Zona}");
    }
    public void LimpiarHabitacion() { }
}
```

Clase hija: Gerente

```
public class Gerente : Empleado
{
    // Atributo: _departamento
    // Propiedad: Departamento

    public Gerente(string identificacion, string departamento)
        : base(identificacion)
    {
        Departamento = departamento;
    }

    public override void ConsultarInformacion()
    {
        Console.WriteLine($"Identificación: {Identificacion}");
        Console.WriteLine($"Soy gerente de {Departamento}");
    }
    public void AutorizarGasto() { }
}
```

Program: ConsultarInformacion()

```
class Program
{
    static void Main()
    {
        Empleado e1 = new Recepcionista("1001", "Inglés");
        Empleado e2 = new PersonalLimpieza("1002", "Piso 3");
        Empleado e3 = new Gerente("1003", "Finanzas");

        e1.ConsultarInformacion();
        e2.ConsultarInformacion();
        e3.ConsultarInformacion();
    }
}
```

Recepcionista con base

```
public class Recepcionista : Empleado
{
    // Atributo: _idioma
    // Propiedad: Idioma

    public Recepcionista(string identificacion, string idioma)
        : base(identificacion)
    {
        Idioma = idioma;
    }

    public override void ConsultarInformacion()
    {
        base.ConsultarInformacion();
        Console.WriteLine($"Idioma: {Idioma}");
    }
    public void AtenderHuesped() { }
}
```

PersonalLimpieza con base

```
public class PersonalLimpieza : Empleado
{
    // Atributo: _zona
    // Propiedad: Zona

    public PersonalLimpieza(string identificacion, string zona)
        : base(identificacion)
    {
        Zona = zona;
    }

    public override void ConsultarInformacion()
    {
        base.ConsultarInformacion();
        Console.WriteLine($"Zona: {Zona}");
    }
    public void LimpiarHabitacion() { }
}
```

Gerente con base

```
public class Gerente : Empleado
{
    // Atributo: _departamento
    // Propiedad: Departamento

    public Gerente(string identificacion, string departamento)
        : base(identificacion)
    {
        Departamento = departamento;
    }

    public override void ConsultarInformacion()
    {
        base.ConsultarInformacion();
        Console.WriteLine($"Departamento: {Departamento}");
    }
    public void AutorizarGasto() { }
}
```

Program: ConsultarInformacion()

```
class Program
{
    static void Main()
    {
        Empleado e1 = new Recepcionista("1001", "Inglés");
        Empleado e2 = new PersonalLimpieza("1002", "Piso 3");
        Empleado e3 = new Gerente("1003", "Finanzas");

        e1.ConsultarInformacion();
        e2.ConsultarInformacion();
        e3.ConsultarInformacion();
    }
}
```

¿Puedo utilizar polimorfismo en las propiedades con nombre común y retornar valores diferentes?

El hotel ahora necesita calcular el salario de cada empleado

Recepcionista: salario base + valor por cada huésped atendido

Personal de limpieza: salario base + valor por cada habitación limpiada

Gerente: salario base + presupuesto sobrante

Salario: virtual y override

```
public class Empleado
    // Atributo: _salarioBase
    public virtual double Salario
    {
        get { return SalarioBase; }
    }
```

```
public class Recepcionista : Empleado
    // Atributo: _huespedesAtendidos
    public void AtenderHuesped()
        => HuespedesAtendidos++;
    public override double Salario
    {
        get { return SalarioBase +
              HuespedesAtendidos * 20000; }
    }
```

```
public class PersonalLimpieza : Empleado
    // Atributo: _habitacionesLimpiadas
    public void LimpiarHabitacion()
        => HabitacionesLimpiadas++;
    public override double Salario
    {
        get { return SalarioBase +
              HabitacionesLimpiadas * 8000; }
    }
```

```
public class Gerente : Empleado
    // Atributo: _presupuesto
    public void AutorizarGasto(double monto)
        => Presupuesto -= monto;
    public override double Salario
    {
        get { return SalarioBase + Presupuesto; }
    }
```

Program: Salario en acción

```
class Program
{
    static void Main()
    {
        Recepcionista recepcionista = new Recepcionista("1001", "Inglés");
        PersonalLimpieza personalLimpieza = new PersonalLimpieza("1002", "Piso 3");
        Gerente gerente = new Gerente("1003", "Finanzas");

        recepcionista.AtenderHuesped();
        personalLimpieza.LimpiarHabitacion();
        gerente.AutorizarGasto(500000);

        Empleado empleado1 = recepcionista;
        Empleado empleado2 = personalLimpieza;
        Empleado empleado3 = gerente;

        Console.WriteLine(empleado1.Salario);
        Console.WriteLine(empleado2.Salario);
        Console.WriteLine(empleado3.Salario);
    }
}
```

¿Y si el comportamiento es común, pero los propiedades son polimórficas?

El salario es común: salario base + una bonificación distinta en cada subclase

Recepcionista: bonificación por cada huésped atendido

Personal de limpieza: bonificación por cada habitación limpiada

Gerente: bonificación igual al presupuesto sobrante

Salario común, bonificación polimórfica

```
public class Empleado
    // Atributo: _salarioBase
    protected virtual double Bonificacion
        { get { return 0; } }
    public double Salario
        { get { return SalarioBase + Bonificacion; } }
```

```
public class Recepcionista : Empleado
    // Atributo: _huespedesAtendidos
    public void AtenderHuesped()
        => HuespedesAtendidos++;
    protected override double Bonificacion
        { get { return HuespedesAtendidos * 20000; } }
```

```
public class PersonalLimpieza : Empleado
    // Atributo: _habitacionesLimpiadas
    public void LimpiarHabitacion()
        => HabitacionesLimpiadas++;
    protected override double Bonificacion
        { get { return HabitacionesLimpiadas * 8000; } }
```

```
public class Gerente : Empleado
    // Atributo: _presupuesto
    public void AutorizarGasto(double monto)
        => Presupuesto -= monto;
    protected override double Bonificacion
        { get { return Presupuesto; } }
```

Program: salario común y bonificación

```
class Program
{
    static void Main()
    {
        Recepcionista recepcionista = new Recepcionista("1001", "Inglés");
        PersonalLimpieza personalLimpieza = new PersonalLimpieza("1002", "Piso 3");
        Gerente gerente = new Gerente("1003", "Finanzas");

        recepcionista.AtenderHuesped();
        personalLimpieza.LimpiarHabitacion();
        gerente.AutorizarGasto(500000);

        Empleado empleado1 = recepcionista;
        Empleado empleado2 = personalLimpieza;
        Empleado empleado3 = gerente;

        Console.WriteLine(empleado1.Salario);
        Console.WriteLine(empleado2.Salario);
        Console.WriteLine(empleado3.Salario);
    }
}
```

Una lista y un solo foreach para todos

```
List<Empleado> empleados = new()
{
    new Recepcionista("1001", "Inglés"),
    new PersonalLimpieza("1002", "Piso 3"),
    new Gerente("1003", "Finanzas")
};

foreach (Empleado empleado in empleados)
{
    empleado.ConsultarInformacion();
    Console.WriteLine(empleado.Salario);
}
```

Actividad: RápidoExpress

La empresa **RápidoExpress** presta servicios de mensajería y transporte de paquetes dentro del territorio nacional. Necesita un sistema para registrar sus envíos, consultar su información y calcular cuánto paga cada cliente. Ofrece tres modalidades: **estándar, exprés y refrigerado.**

Todo envío tiene **número de guía, nombre del destinatario, peso en kilogramos y tarifa base de transporte.**

Estándar

- Mercancías sin condiciones especiales.
- **Costo: tarifa base + $2.000 por kg.**

Exprés

- Entrega con mayor prioridad.
- Registra el plazo máximo de entrega (horas).
- **Costo: base + $2.000/kg + $15.000.**

Refrigerado

- Productos a temperatura controlada.
- Registra la temperatura requerida (°C).
- **Costo: base + $2.000/kg + $25.000.**

Cualquier envío debe **consultar su información general y su modalidad**, con sus datos particulares. La empresa quiere **consultar los costos de un conjunto de envíos** sin un procedimiento distinto para cada modalidad.